

Protocole de constitution et de traitement du corpus

Marc Garnier

« *That's kino anon* » : les charts de /lit/ comme dispositifs gamifiés.

D'une lecture qualitative à une mesure computationnelle du corpus.

Archive de données — <https://osf.io/hrb26/> (corpus d'images).

Code, lexiques, données dérivées, carnet d'analyse — <https://github.com/marcgarnier/lit-charts>.

Version du protocole — 1.0, septembre 2026. Il décrit l'état du pipeline tel qu'il a produit les résultats publiés ; toute révision ultérieure du code est tracée par l'historique du dépôt.

Ce document est le protocole : il énonce comment le corpus a été constitué, ce que chaque étape de traitement fait, avec quels paramètres, ce qui a été mesuré, ce qui a été rejeté et pourquoi. Il est écrit pour qu'un tiers puisse rejouer la chaîne complète, ou en contester un maillon précis, sans avoir à lire le code.

1. Ce que contient l'archive et où trouver le reste

Élément	Emplacement
Corpus d'images (264 charts)	archive OSF, <code>database.zip</code>
Ce protocole	archive OSF et dépôt de code
Code du pipeline, commenté en français	dépôt, <code>src/</code>
Lexiques et prompts versionnés	dépôt, <code>prompts/</code>
Données dérivées (extractions, tables)	dépôt, <code>data/interim/</code> , <code>data/processed/</code>
Rapports de fiabilité et de nettoyage	dépôt, <code>results/</code>
Carnet d'analyse exécuté, figures incluses	dépôt, <code>notebooks/resultats.ipynb</code>
Article	dépôt, <code>papier/</code>
Décisions de méthode, y compris les échecs mesurés	dépôt, <code>METHODE.md</code>

Les images sont séparées du code pour des raisons de volume ; le dépôt versionne en revanche toutes les données dérivées, de sorte que chaque maillon — de l'image brute au chiffre publié — peut être inspecté ou rejoué indépendamment.

2. Corpus

Source. Le corpus est constitué des 264 images rassemblées par le wiki communautaire des charts du board /lit/ de 4chan : la base de 262 charts réunie pour le travail qualitatif initial, complétée de deux doublons de classement (une même image rangée dans deux catégories thématiques du wiki, conservés pour ne pas perdre l'information de rangement multiple).

Classement indigène. Le wiki range les images en onze catégories thématiques (Beginnings, General, Philosophy, Countries, Speculative Fiction, Religion, Ideologies, Pills, Science, Meme Charts, Other Boards). Ce classement est conservé comme **point de comparaison externe uniquement** : il n'entre jamais comme variable dans la construction de la typologie, ce qui serait circulaire.

Hétérogénéité matérielle. Les images vont de 0,3 à 43,1 mégapixels (médiane 4,1), jusqu'à 7 600 px de côté. Cette dispersion commande trois choix de méthode : l'OCR est appliqué par bandes et non à l'image entière (§ 4.1), les seuils géométriques sont exprimés en unités relatives à chaque image et non en pixels absolus (§ 4.3), et les images envoyées au modèle vision-langage sont réduites (§ 4.2) pour tenir dans 8 Go de mémoire.

Deux limites de constitution, posées d'emblée. (i) Le corpus provient du wiki et non des fils de discussion : il ne comporte **aucune métadonnée de circulation** (date de publication, nombre de reposts) ; toute analyse de recirculation porte donc sur le contenu des charts, non sur leur diffusion observée. (ii) Le wiki opère sa propre curation : le corpus représente les charts jugés dignes d'archivage par la communauté, non l'ensemble des charts ayant circulé. Un module de collecte directe (`src/01_collect.py`, API JSON publique de 4chan et archive FoolFuuka) existe dans le dépôt, mais l'archive historique de /lit/ n'exposant pas d'API exploitable, il reste une extension et n'est pas la source du présent corpus.

3. Principes d'exécution

Local de bout en bout. Aucune API distante, aucune clé, aucun service payant : la chaîne complète tourne sur une machine personnelle ordinaire (Apple M1, 8 Go de mémoire unifiée, macOS). Ce n'est pas seulement un parti pris économique : il garantit qu'un tiers peut tout rejouer sans dépendre d'un service susceptible de changer, d'être facturé ou d'être retiré.

Relançabilité sans recalcul. Chaque script vérifie l'existence de ses sorties et saute ce qui est déjà fait ; l'option `--force` retraite. Un traitement long (OCR, classement visuel) peut donc être interrompu et repris sans perte.

Déterminisme. Toutes les graines aléatoires sont fixées (`k-moyennes random_state = 42`, disposition du graphe `seed = 42`). Les étapes non stochastiques — géométrie, relevé lexical, nettoyage des auteurs — rendent le même résultat à chaque exécution.

Traçabilité. Le moteur d'OCR employé est inscrit dans chaque fichier produit ; chaque détection lexicale conserve la ligne qui la justifie ; la provenance du codage des formes figure en tête du fichier de données ; les rapports de nettoyage et d'accord inter-codeurs sont versionnés avec le code.

Règle d'arbitrage entre outils, établie par mesure et non par principe : ne jamais demander à un modèle ce qu'un calcul déterministe peut établir, ni à un modèle de vision ce qui relève du texte. Les mesures qui ont conduit à cette règle sont consignées en § 4 et dans `METHODE.md`.

4. Chaîne de traitement

Ordre logique d'exécution — la numérotation des fichiers reflète l'ordre historique d'écriture, non l'ordre d'appel :

```
01_ocr.py → 03_vlm.py (36 charts, mesure d'accord) → 02_blocs.py
→ 06_marqueurs.py → 05_tables.py → 07_traits.py
→ 08_accord.py → 09_auteurs.py → notebooks/resultats.ipynb
```

4.1 Extraction du texte par OCR (*01_ocr.py*)

Chaque image est découpée en **bandes horizontales lues à pleine résolution**, puis les résultats sont fusionnés en écartant les doublons de recouvrement (même texte à quelques pixels près). Sur un chart de contrôle en grille (« /lit/s Top 100 Books »), l'OCR plein format relevait 2 titres sur 9 vérifiés, contre 9 sur 9 après découpage.

Deux moteurs interchangeables : le framework **Vision** de macOS par défaut (accéléré matériellement, propriétaire, limité à macOS) et **Surya 0.16.7**, libre et multiplateforme, comme contrôle. Surya reproduit les mêmes extractions sur les vérités terrain (14 sections sur 14 et 185 œuvres sur une carte de philosophie ; 9 titres sur 9 et 8 auteurs sur 8 sur la grille « Top 100 ») pour un coût temporel environ 75 fois supérieur. Passer --moteur surya sur un échantillon atteste que les résultats ne dépendent pas d'un composant fermé.

Sortie : **34 581 segments** de texte non vides, chacun avec sa boîte englobante ramenée au repère de l'image d'origine et un score de confiance ; le balisage de mise en forme éventuel est conservé dans un champ séparé, un champ normalisé servant aux appariements ultérieurs. Temps : 10,9 minutes pour les 264 images avec Vision (2,5 s par chart en moyenne, 15,5 s au maximum), sans erreur.

4.2 Classement visuel par modèle vision-langage (*03_vlm.py*)

Une seule question, après l'OCR, relève encore de la vision : la forme d'ensemble du chart et la présence de liens dessinés. Modèle **Qwen2.5-VL 3B servi localement par Ollama**, prompt versionné (prompts/vlm.txt) lui interdisant explicitement de lire ou transcrire le texte, sortie contrainte en JSON, **température nulle**, image réduite à **500 px de côté** (à cette taille la forme reste lisible, le texte ne l'est plus : temps d'inférence divisé par sept, réponse de forme inchangée).

Échec mesuré, conservé au dossier. Une première version demandait au même modèle d'extraire aussi les titres et la structure : sur « /lit/s Top 100 Books » il rendait 5 livres au lieu de 100, avec des rangs faux et des flèches imaginaires, en onze minutes par image et en saturation mémoire. Restreint à la forme, il répond juste et vite. Mais même ainsi, sa classification multi-classes de la mise en page s'est révélée non fiable à la mesure (§ 6) et a été **rejetée** ; seule la variable binaire « présence de flèches » est conservée. Le classement visuel n'a donc été calculé que sur les 36 charts nécessaires à la mesure d'accord ; ailleurs, la géométrie choisit son régime par heuristique de secours.

4.3 Reconstruction géométrique de la structure (*02_blocs.py*)

Aucun modèle : les coordonnées produites par l'OCR contiennent déjà la réponse. Les lignes sont regroupées en blocs, chaque bloc rattaché à l'intitulé qui le gouverne, et chaque nœud décrit au format {titre, auteur, tier}.

Deux régimes de reconstruction, selon que le chart est « en texte » (listes, sections) ou « à couvertures » (grilles régulières de vignettes). Le régime suit le layout_type de la phase 4.2 quand il est disponible ; sinon, une **heuristique géométrique de secours** tranche : une grille se signale par des bandes horizontales de population régulière et des colonnes équidistantes (coefficient de variation des écarts inférieur à 0,20 — mesuré à 0,10 sur la grille « Top 100 » contre 0,31 sur la carte de philosophie). Sur les tier lists à couvertures, un détecteur d'intitulés isolés récupère les paliers nommés (« God-Tier », « Entry-level Tier »), qui portent l'essentiel de la gamification.

Contrôles : la procédure retrouve les 14 grappes et leurs 185 œuvres de la carte de philosophie, et apparie exactement 100 nœuds sur la grille « Top 100 ». Les sorties sont consolidées avec l'inventaire

du wiki en deux tables (05_tables.py) : charts.csv (une ligne par chart) et oeuvres.csv (une ligne par couple chart-œuvre).

4.4 Relevé lexical des marqueurs de gamification (06_marqueurs.py)

Neuf registres relevés par expressions régulières, insensibles à la casse, dans le texte intégral de chaque chart : **progression, injonction, point d'entrée, prérequis, difficulté graduée, optionnel, récompense, paliers nommés, hiérarchie de valeur.**

Le lexique est un fichier autonome et versionné (prompts/marqueurs.json) : c'est l'instrument de mesure de l'étude, il doit pouvoir être amendé et rejoué indépendamment du code. **Chaque détection conserve la ligne qui la justifie**, ce qui rend le relevé vérifiable à la main et expose ses erreurs — sur un chart consacré à Dante, les 27 « injonctions » détectées se sont révélées être des vers cités de la *Commedia*, erreur identifiable précisément parce que la preuve est conservée.

Voie alternative écartée. Un codage des mécaniques par LLM de texte local (Llama 3.1 8B via Ollama) est implémenté (04_codage.py) et reste dans le dépôt comme extension, mais n'a pas été retenu : le relevé lexical déterministe lui a été préféré parce qu'il est reproductible au caractère près, vérifiable ligne à ligne, et sans dépendance à la variabilité d'un modèle génératif.

4.5 Traits mesurables et typologie émergente (07_traits.py, carnet)

Chaque chart est décrit par **19 variables** entièrement dérivées des sorties précédentes : dix variables de forme issues de l'OCR (proportions de l'image, densité de texte, couverture verticale, part de bandes horizontales peuplées, régularité des colonnes, dispersion des hauteurs de ligne, longueur moyenne et part de lignes courtes, confiance médiane) et les neuf registres lexicaux. Deux colonnes de contrôle — la forme codée (§ 5) et la catégorie du wiki — figurent dans la table mais **n'entrent jamais** dans le calcul des groupes.

Les variables de comptage sont passées au logarithme pour amortir leur asymétrie, puis l'ensemble est standardisé. Classification par **k-moyennes** (scikit-learn, `n_init = 10`, graine fixée), nombre de groupes choisi au **critère de silhouette** : optimum net à $k = 2$ (0,309, contre environ 0,12 pour tout $k \geq 3$).

Validation externe. La partition est confrontée par l'**indice de Rand ajusté** à trois classements indépendants : la forme codée, les catégories thématiques du wiki, et la typologie théorique en trois types approchée depuis les formes. C'est le test central : la typologie par les mécaniques est-elle réductible à la forme visuelle ou au sujet ?

4.6 Réseau des auteurs (09_auteurs.py, carnet)

Les mentions d'auteur proviennent du motif « Auteur : Titre » appliqué au texte OCR : **8 470 mentions**, soit 24,5 % des segments — les charts figurant les livres par leur seule couverture ne livrent aucun nom, ce qui borne le réseau à ce qui est textuellement énoncé.

Deux bruits sont traités. Les **faux auteurs** (« History: A Very Short Introduction ») : le critère le plus discriminant s'est révélé être la présence d'un mot grammatical anglais dans la chaîne, qu'un fragment de titre contient presque toujours et un nom de personne jamais ; les particules nobiliaires enchâssées sont préservées pour ne pas casser « Simone de Beauvoir ». Les **formes éclatées** (« Kafka », « Franz Kafka », « KAFKA ») : rapprochement par patronyme (dernier mot, sans accents ni casse) puis par similarité de chaînes pour les coquilles d'OCR (« Dostoyevsky » → « Dostoevsky »), un garde-fou empêchant la fusion des paires singulier/pluriel. Un lexique d'exclusions versionné (prompts/auteurs_exclusions.txt) recueille les faux auteurs résiduels repérés à l'œil.

Résultat : **5 518 mentions** pour **3 006 auteurs distincts**, chaque forme écartée et chaque fusion étant consignées dans `results/auteurs_rapport.txt`. Deux auteurs sont liés s'ils figurent sur un même chart ; le graphe de cooccurrence est construit avec NetworkX sur le **noyau des auteurs présents dans au moins cinq charts**.

5. Codage de référence des formes

Le codage de la forme des 264 charts (`data/interim/codage_manuel_formes.csv`) n'a été réalisé **ni par un humain ni par le pipeline scripté** : il a été produit par **Molmo 2 (Ai2)**, modèle vision-langage à poids ouverts exécuté localement, qui a examiné chaque image sur planches contact, avec vérifications ponctuelles en pleine résolution.

Taxonomie construite **inductivement** sur le corpus, en sept modalités : grille simple, grille à sections nommées, organigramme, tier list, liste de texte, collage, carte. Deux modalités — la grille à sections, et la distinction tier list / grille à sections selon que l'intitulé exprime un rang ou un thème — ont dû être ajoutées en cours de codage, la taxonomie initiale ne rendant pas compte du corpus.

Statut : validation externe. Ce codage sert à juger les partitions émergentes, au même titre que les catégories du wiki, et reste extérieur au pipeline reproductible pour deux raisons qu'il faut énoncer : aucun script du dépôt ne le produit, et il a servi à construire la taxonomie, ce qui l'avantage structurellement comme référence. Sa provenance est déclarée en tête du fichier de données lui-même. L'idéal méthodologique — un recodage d'échantillon par l'auteur, permettant de rapporter un accord humain-machine — est une extension prévue et non réalisée à ce jour.

6. Fiabilité : mesure et décision de rejet

L'accord entre le classement visuel du modèle local (§ 4.2) et le codage de référence (§ 5) est mesuré par l' **α de Krippendorff** (`08_accord.py`, rapport dans `results/accord_inter_codeurs.txt`), indice retenu plutôt que le kappa de Cohen parce qu'il accepte plus de deux annotateurs, des catégories multiples et des données manquantes. Il rapporte le désaccord observé au désaccord attendu si les étiquettes étaient distribuées au hasard selon les fréquences de chaque codeur : 1 pour l'accord parfait, 0 pour un accord équivalent au hasard, négatif en deçà.

Seuils retenus (Krippendorff) : $\alpha \geq 0,800$ fiable, conclusions permises ; $0,667-0,800$ acceptable pour des conclusions provisoires ; $< 0,667$ insuffisant, la variable ne doit pas être exploitée.

Variable	Charts communs	α	Décision
layout_type (nominal, 7 modalités)	36	0,078	rejetée — sous tout seuil
has_arrows (binaire)	36	0,842	conservée — au-dessus du seuil de fiabilité

Le diagnostic est net : le modèle local répond « grille à sections » 27 fois sur 36, ne discriminant presque pas (accord brut 39 %, contre 33 % pour un annotateur constant qui répondrait toujours la même modalité sans regarder les images). Un petit modèle local traite correctement une question perceptive binaire et échoue sur une catégorisation abstraite à sept modalités : ce résultat négatif est conservé et publié plutôt que masqué.

7. Reproduire la chaîne

```
conda create -n lit-charts python=3.11 -c conda-forge --override-channels
conda activate lit-charts
pip install -r requirements.txt
# phases à modèles locaux : installer Ollama (https://ollama.com) puis
ollama pull qwen2.5vl:3b && ollama pull llama3.1
```

Télécharger `database.zip` depuis l'archive OSF, décompresser les images dans `data/raw/images/`, puis dérouler les scripts dans l'ordre du § 4. Le carnet `notebooks/resultats.ipynb`, versionné avec ses sorties, produit l'ensemble des figures et des chiffres de l'article.

Dépendances principales : Pillow ; liaisons PyObjC vers le framework Vision d'Apple (OCR par défaut, macOS) ; Surya 0.16.7 avec transformers 4.x (OCR de contrôle, multiplateforme) ; Ollama servant Qwen2.5-VL 3B et Llama 3.1 8B ; NumPy, pandas, scikit-learn, NetworkX, Matplotlib. Versions exactes figées dans `requirements.txt`.

Volumétrie et temps (corpus de 264 charts, Apple M1 8 Go) :

Étape	Sortie	Temps
OCR (Vision)	34 581 segments	10,9 min (2,5 s/chart)
OCR (Surya, contrôle)	idem, vérités terrain	≈ 5 min/chart
Classement visuel (VLM)	36 charts classés	≈ 19 s/chart (à froid)
Géométrie	blocs, sections, nœuds	< 1 s/chart
Marqueurs lexicaux	9 registres + preuves	quasi instantané
Traits	matrice 264 × 19	quasi instantané
Nettoyage des auteurs	8 470 → 5 518 mentions	quasi instantané
Typologie, réseau	groupes, graphe	quelques secondes

8. Limites déclarées

- Couverture des auteurs** — 24,5 % des segments portent un auteur identifiable ; les charts à couvertures muettes sont sous-représentés dans le réseau, qui décrit le canon *textuellement* énoncé.
- Provenance du codage des formes** — réalisé hors pipeline par un modèle vision-langage bien plus grand que celui du pipeline (§ 5) : validation externe déclarée, non vérité terrain humaine.
- Absence de données de circulation** — le corpus vient du wiki ; la recirculation n'est approchée que par le contenu, et le corpus reflète la curation du wiki plutôt que l'ensemble des charts ayant circulé.
- Dépendance au volume de texte** — les scores lexicaux croissent mécaniquement avec la quantité de texte ; les comparaisons portent sur des taux de présence, toute exploitation fine des intensités exigerait une normalisation.
- Taxonomie et effectifs faibles** — les modalités « carte » (3 charts) et « collage » (11) sont trop peu peuplées pour un traitement statistique séparé, et la taxonomie a dû être révisée en cours de codage, ce qui signale son caractère inductif et perfectible.

9. Données, éthique et diffusion

Le corpus est composé d'**artefacts publics d'utilisateurs anonymes** — des images-guides mises en circulation sur un board anonyme, puis archivées par un wiki communautaire — collectés à des fins d'analyse académique. Aucune donnée personnelle, aucun identifiant d'utilisateur, aucune métadonnée de fil n'est collectée ni redistribuée : l'unité d'analyse est l'image et son contenu textuel, jamais son auteur. Les charts ne sont pas reproduits individuellement dans l'article au-delà de ce qu'exige la démonstration méthodologique.

Disponibilité. Images : <https://osf.io/hrb26/>. Code, lexiques, données dérivées, rapports et carnet exécuté : <https://github.com/marcgarnier/lit-charts>. Le code et les lexiques sont sous licence MIT ; l'article est © Marc Garnier.

Citation. Garnier, M. (2026). « *That's kino anon* » : les charts de /lit/ comme dispositifs gamifiés. D'une lecture qualitative à une mesure computationnelle du corpus. <https://github.com/marcgarnier/lit-charts>